

---

# Academic Events API

Relatório Técnico - Trabalho Final

Módulo 3: Desenvolvimento em C#

---

**Disciplina:** Desenvolvimento em C# - Módulo 3

**Responsável:** Juliana Balllin

**Data:** Junho de 2026

## Autoria do Relatório

1. Juliana Balllin

# Sumário

|           |                                                       |          |
|-----------|-------------------------------------------------------|----------|
| <b>1</b>  | <b>Introdução</b>                                     | <b>2</b> |
| <b>2</b>  | <b>Problema e Solução</b>                             | <b>2</b> |
| 2.1       | O Problema . . . . .                                  | 2        |
| 2.2       | Nossa Solução . . . . .                               | 2        |
| <b>3</b>  | <b>Tecnologias Utilizadas</b>                         | <b>2</b> |
| <b>4</b>  | <b>Arquitetura do Sistema</b>                         | <b>3</b> |
| 4.1       | Visão Geral - Clean Architecture em Camadas . . . . . | 3        |
| 4.2       | Fluxo de Dependências entre Projetos . . . . .        | 3        |
| 4.3       | Por que essa Arquitetura? . . . . .                   | 3        |
| <b>5</b>  | <b>Modelagem do Domínio</b>                           | <b>3</b> |
| 5.1       | Entidades . . . . .                                   | 3        |
| 5.2       | Enumeradores . . . . .                                | 4        |
| <b>6</b>  | <b>Autenticação e Autorização</b>                     | <b>4</b> |
| 6.1       | JWT Bearer Token . . . . .                            | 4        |
| 6.2       | Configuração do Token . . . . .                       | 4        |
| 6.3       | Hash de Senha . . . . .                               | 5        |
| <b>7</b>  | <b>Endpoints da API</b>                               | <b>5</b> |
| 7.1       | Autenticação (público) . . . . .                      | 5        |
| 7.2       | Eventos . . . . .                                     | 5        |
| 7.3       | Inscrições, Comentários e Reações . . . . .           | 5        |
| 7.4       | Formato dos Enums nos Contratos . . . . .             | 6        |
| <b>8</b>  | <b>Camada de Infrastructure</b>                       | <b>6</b> |
| 8.1       | DbContext e Configurações EF Core . . . . .           | 6        |
| 8.2       | Repository Pattern . . . . .                          | 6        |
| 8.3       | Banco de Dados via Docker . . . . .                   | 6        |
| <b>9</b>  | <b>Regras de Negócio Implementadas</b>                | <b>7</b> |
| <b>10</b> | <b>Dificuldades e Decisões Técnicas</b>               | <b>7</b> |
| 10.1      | Versões dos Pacotes NuGet . . . . .                   | 7        |
| 10.2      | Forbid() vs StatusCode(403) . . . . .                 | 7        |
| 10.3      | Application não depende de Infrastructure . . . . .   | 7        |
| 10.4      | Validação em Dois Níveis . . . . .                    | 7        |
| <b>11</b> | <b>Como Executar o Projeto</b>                        | <b>8</b> |
| <b>12</b> | <b>Testes Automatizados e Integração Contínua</b>     | <b>8</b> |
| <b>13</b> | <b>Conclusão</b>                                      | <b>8</b> |

## 1 Introdução

O presente relatório descreve o desenvolvimento da **Academic Events API**, uma API REST criada como trabalho final do Módulo 3 do curso de Desenvolvimento em C#. O projeto implementa um sistema de gerenciamento de eventos acadêmicos, como palestras, workshops e seminários, seguindo as boas práticas de desenvolvimento com ASP.NET Core 8.

O objetivo principal foi aplicar os conceitos ensinados ao longo do módulo: orientação a objetos, DTOs, services, repositories, acesso a banco de dados, autenticação e autorização, e organização profissional de código em camadas separadas.

A plataforma permite que usuários se cadastrem, criem e gerenciem eventos acadêmicos, realizem inscrições, publiquem comentários e expressem reações a conteúdos de interesse, funcionando de forma análoga a uma rede social voltada para o ambiente acadêmico.

## 2 Problema e Solução

### 2.1 O Problema

Eventos acadêmicos, como palestras, workshops, minicursos e seminários, são parte essencial da vida universitária. No entanto, a organização e divulgação desses eventos muitas vezes acontece de forma fragmentada: grupos de WhatsApp, cartazes físicos e e-mails avulsos dificultam o acompanhamento das oportunidades disponíveis e o controle de inscrições.

### 2.2 Nossa Solução

A Academic Events API centraliza o ciclo de vida completo de um evento acadêmico em uma única plataforma, expondo endpoints REST documentados via Swagger. Qualquer aplicação cliente, seja web, mobile ou desktop, pode consumir a API para:

- Listar e filtrar eventos por status ou organizador
- Criar e gerenciar eventos (organizadores autenticados)
- Realizar e acompanhar inscrições
- Comentar e reagir a eventos de interesse

## 3 Tecnologias Utilizadas

| Tecnologia            | Versão | Finalidade                            |
|-----------------------|--------|---------------------------------------|
| ASP.NET Core          | 8.0    | Framework principal da API REST       |
| Entity Framework Core | 8.0.11 | ORM para acesso ao banco de dados     |
| Npgsql (PostgreSQL)   | 8.0.11 | Driver de conexão com PostgreSQL      |
| JWT Bearer Token      | 8.0.15 | Autenticação e autorização via token  |
| BCrypt.Net-Next       | 4.2.0  | Hash seguro de senhas dos usuários    |
| Swashbuckle / Swagger | 6.6.2  | Documentação automática dos endpoints |
| PostgreSQL            | 15     | Banco de dados relacional (Docker)    |
| Docker Compose        | -      | Ambiente de banco de dados isolado    |

## 4 Arquitetura do Sistema

### 4.1 Visão Geral - Clean Architecture em Camadas

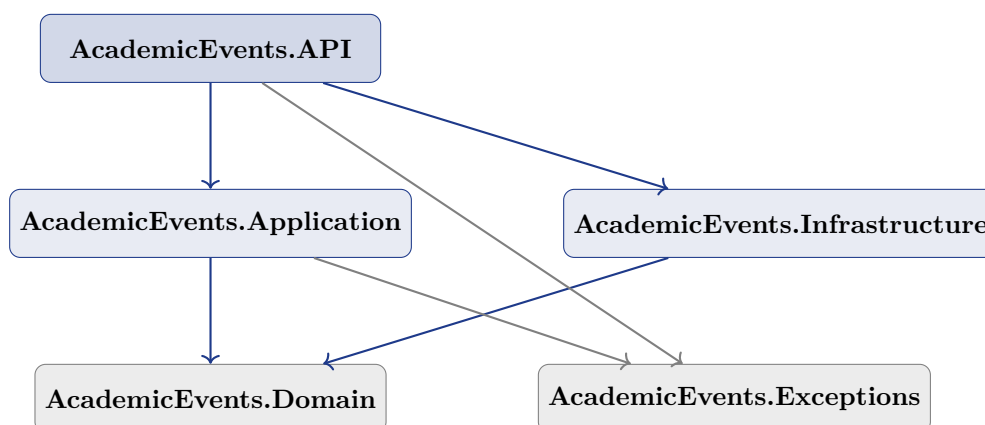
O projeto é organizado em cinco projetos C# separados dentro de uma única solution, seguindo os princípios de separação de responsabilidades e inversão de dependência:

#### Estrutura da Solution

- **AcademicEvents.Domain** - Entidades, enums, interfaces de repository (sem dependências externas)
- **AcademicEvents.Exceptions** - Exceções customizadas com semântica de negócio
- **AcademicEvents.Application** - DTOs, interfaces de service, implementações de service
- **AcademicEvents.Infrastructure** - DbContext, EF Core, implementações de repository
- **AcademicEvents.API** - Controllers, JWT, Swagger, Program.cs (ponto de entrada)

### 4.2 Fluxo de Dependências entre Projetos

O gráfico abaixo ilustra como os projetos se relacionam, respeitando a regra de que as camadas internas nunca dependem das externas:



### 4.3 Por que essa Arquitetura?

- **Domain sem dependências:** as entidades e interfaces ficam isoladas, podendo ser testadas sem infraestrutura
- **Application só depende de Domain:** os services não sabem como os dados são persistidos. Apenas chamam interfaces
- **Infrastructure implementa interfaces do Domain:** a inversão de dependência permite trocar o banco (ex: SQLite para testes) sem mudar a lógica de negócio
- **API orquestra tudo via DI:** o Program.cs conecta as implementações concretas usando extension methods de injeção de dependência

## 5 Modelagem do Domínio

### 5.1 Entidades

O domínio é composto por cinco entidades principais com os respectivos relacionamentos:

| Entidade     | Campos principais                                                        | Descrição                                                                                    |
|--------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| User         | Id, Nome, Email, SenhaHash, CriadoEm                                     | Usuário da plataforma. Pode organizar eventos, se inscrever, comentar e reagir               |
| Event        | Id, Titulo, Descricao, DataInicio, DataFim, Local, Status, OrganizadorId | Evento acadêmico criado por um organizador. Ciclo: Rascunho, Publicado, Cancelado, Concluído |
| Registration | Id, UsuarioId, EventoId, Status                                          | Inscrição de um usuário em um evento. Índice único impede duplicatas                         |
| Comment      | Id, UsuarioId, EventoId, Conteudo                                        | Comentário do usuário em um evento                                                           |
| Reaction     | Id, UsuarioId, EventoId, Tipo                                            | Reação (curtida) de um usuário a um evento. Um usuário só pode ter uma reação por evento     |

## 5.2 Enumeradores

| Enum            | Valores                                     |
|-----------------|---------------------------------------------|
| StatusEvento    | Rascunho, Publicado, Cancelado, Concluído   |
| StatusInscricao | Pendente, Confirmada, Cancelada             |
| TipoReacao      | Curtir, Adorei, Interessante, VouParticipar |

## 6 Autenticação e Autorização

### 6.1 JWT Bearer Token

A autenticação utiliza o padrão **JWT (JSON Web Token)** com algoritmo HMAC-SHA256. O fluxo é:

1. Usuário envia email e senha para `POST /api/auth/login`
2. O `AuthService` valida as credenciais com `BCrypt`
3. Um token JWT é gerado com as *claims*: `NameIdentifier` (id), `Email` e `Name`
4. O token é retornado e deve ser enviado no header `Authorization: Bearer {token}`
5. O middleware do ASP.NET valida o token automaticamente em todas as rotas com `[Authorize]`

### 6.2 Configuração do Token

```
// appsettings.json
"Jwt": {
  "Key": "chave-secreta-mínimo-32-caracteres-aqui",
  "Issuer": "AcademicEventsAPI",
  "Audience": "AcademicEventsClientes",
  "ExpiresInHours": 8
}
```

### 6.3 Hash de Senha

As senhas dos usuários nunca são armazenadas em texto puro. O **BCrypt** é usado para gerar um hash com salt automático, tornando ataques de dicionário impraticáveis mesmo que o banco seja comprometido.

## 7 Endpoints da API

### 7.1 Autenticação (público)

| Método | Rota               | Retorno  | Descrição                          |
|--------|--------------------|----------|------------------------------------|
| POST   | /api/auth/register | 200, 400 | Cadastra usuário e retorna JWT     |
| POST   | /api/auth/login    | 200, 401 | Autentica usuário e retorna JWT    |
| GET    | /api/me            | 200, 401 | Dados do usuário autenticado (JWT) |

### 7.2 Eventos

| Método | Rota                                 | Auth | Descrição                               |
|--------|--------------------------------------|------|-----------------------------------------|
| GET    | /api/events                          | Não  | Lista todos os eventos                  |
| GET    | /api/events?status=X                 | Não  | Filtra por status                       |
| GET    | /api/events/{id}                     | Não  | Busca evento por id                     |
| GET    | /api/events?organizadorId=X          | Não  | Filtra por organizador                  |
| GET    | /api/events?status=X&organizadorId=Y | Não  | Combina filtros de status e organizador |
| GET    | /api/events/meus                     | Sim  | Eventos do organizador autenticado      |
| POST   | /api/events                          | Sim  | Cria novo evento                        |
| PUT    | /api/events/{id}                     | Sim  | Atualiza evento (só o organizador)      |
| DELETE | /api/events/{id}                     | Sim  | Remove evento (só o organizador)        |

### 7.3 Inscrições, Comentários e Reações

| Método | Rota                     | Auth | Descrição                      |
|--------|--------------------------|------|--------------------------------|
| POST   | /api/registrations       | Sim  | Inscreve usuário em evento     |
| GET    | /api/registrations/me    | Sim  | Minhas inscrições              |
| DELETE | /api/registrations/{id}  | Sim  | Cancela inscrição              |
| GET    | /api/comments?eventoId=X | Não  | Comentários de um evento       |
| POST   | /api/comments            | Sim  | Adiciona comentário            |
| DELETE | /api/comments/{id}       | Sim  | Remove comentário (só o autor) |

| Método | Rota                      | Auth | Descrição                  |
|--------|---------------------------|------|----------------------------|
| GET    | /api/reactions?eventoId=X | Não  | Reações de um evento       |
| POST   | /api/reactions            | Sim  | Adiciona reação            |
| DELETE | /api/reactions/{id}       | Sim  | Remove reação (só o autor) |

## 7.4 Formato dos Enums nos Contratos

O projeto configura o `JsonStringEnumConverter` no `Program.cs`. Com isso, os campos de enum podem ser enviados como texto no corpo das requisições, mantendo os exemplos do Swagger e do arquivo `endpoints.http` mais legíveis. Exemplos: "Publicado" para `StatusEvento` e "VouParticipar" para `TipoReacao`.

## 8 Camada de Infrastructure

### 8.1 DbContext e Configurações EF Core

O `AcademicEventsDbContext` configura os mapeamentos via *Fluent API* no método `OnModelCreating`:

- **Índice único em `User.Email`**: impede dois usuários com o mesmo email
- **Índice único em (`UsuarioId`, `EventoId`) na `Registration`**: impede inscrição duplicada
- **Índice único em (`UsuarioId`, `EventoId`) na `Reaction`**: impede reação duplicada
- **`DeleteBehavior.Restrict` em `Event` para `User`**: impede deletar usuário que tem eventos
- **`DeleteBehavior.Cascade` nos demais**: ao remover usuário ou evento, registros dependentes são apagados

### 8.2 Repository Pattern

Cada entidade tem seu próprio repository que abstrai o acesso ao banco:

```
// exemplo de consulta com Include para evitar N+1 queries
public async Task<List<Event>> GetAllAsync()
{
    return await _context.Events
        .Include(e => e.Organizador)
        .OrderByDescending(e => e.DataInicio)
        .ToListAsync();
}
```

O uso de `Include` garante que o nome do organizador seja carregado junto com o evento em uma única query SQL, evitando o problema N+1.

### 8.3 Banco de Dados via Docker

O ambiente de banco de dados é provisionado via `docker-compose.yml`:

```
docker compose up -d # sobe o PostgreSQL 15 na porta 5432
```

A inicialização do schema usa `EnsureCreated()` no startup da API, criando as tabelas automaticamente se não existirem.

## 9 Regras de Negócio Implementadas

| Regra                                     | Implementação                                                                             |
|-------------------------------------------|-------------------------------------------------------------------------------------------|
| Email único                               | Verificado no AuthService antes de inserir no banco (400 BadRequest)                      |
| Senha segura (min 6 chars)                | Validação de DataAnnotations no RegisterRequest                                           |
| Credenciais inválidas                     | Mesma mensagem independente de email ou senha errados (segurança)                         |
| DataFim maior que DataInicio              | Validado no EventService ao criar e atualizar evento (400 BadRequest)                     |
| Só organizador edita evento               | Verificação de OrganizadorId no EventService (403 Forbidden)                              |
| Inscrição duplicada                       | Verificação no RegistrationService antes do banco (400 BadRequest)                        |
| Evento existente em inscrição             | RegistrationService retorna 404 quando o evento informado não existe                      |
| Reação única por evento                   | Verificação no ReactionService antes do banco (400 BadRequest)                            |
| Evento existente em comentários e reações | CommentService e ReactionService retornam 404 quando o evento informado não existe        |
| Só autor deleta comentário                | Verificação de UsuarioId no CommentService (403 Forbidden)                                |
| Respostas HTTP corretas                   | 200/201 sucesso, 400 dados inválidos, 401 sem auth, 403 sem permissão, 404 não encontrado |

## 10 Dificuldades e Decisões Técnicas

### 10.1 Versões dos Pacotes NuGet

A combinação de versões dos pacotes do Entity Framework Core para .NET 8 exigiu atenção especial. A versão 8.0.11 do `Npgsql.EntityFrameworkCore.PostgreSQL` foi escolhida por ser a mais estável e compatível com o .NET 8 e o `Microsoft.EntityFrameworkCore.Design` 8.0.11.

### 10.2 Forbid() vs StatusCode(403)

O método `Forbid()` do `ControllerBase` do ASP.NET Core não aceita uma mensagem de texto. Ele espera um nome de esquema de autenticação. Para retornar 403 com mensagem de erro no corpo da resposta, o correto é usar `StatusCode(403, mensagem)`.

### 10.3 Application não depende de Infrastructure

A camada Application só referencia o Domain (onde estão as interfaces de repository). A Infrastructure é registrada no DI pela API via extension method, resolvendo as implementações concretas em tempo de execução. Isso garante a inversão de dependência correta.

### 10.4 Validação em Dois Níveis

Para inscrições e reações duplicadas, a validação acontece tanto no service (verificação prévia antes do banco) quanto no banco (índice único). O service lança uma exceção customizada com mensagem amigável; o banco é a última linha de defesa contra condições de corrida.

## 11 Como Executar o Projeto



- .NET 8 SDK
- Docker e Docker Compose
- (Opcional) dotnet-ef para migrations: `dotnet tool install --global dotnet-ef`

```
# 1. clonar o repositório
git clone https://github.com/JulianaBallin/Academic-Events-API.git
cd Academic-Events-API

# 2. subir o banco PostgreSQL via Docker
docker compose up -d

# 3. iniciar a API (o EnsureCreated cria as tabelas automaticamente)
cd AcademicEvents.API
dotnet run
```

Após iniciar, acesse a documentação interativa no Swagger em: <http://localhost:5000/swagger>

Para testar os endpoints com autenticação, clique em **Authorize** no Swagger e informe o token obtido no login no formato `Bearer {token}`.

## 12 Testes Automatizados e Integração Contínua

O projeto também possui o projeto `AcademicEvents.Tests`, criado com xUnit e Moq para validar as regras de negócio da camada Application sem depender do PostgreSQL. Essa abordagem permite testar os services diretamente, usando mocks das interfaces de repository definidas no Domain.

```
dotnet test AcademicEvents.sln
```

| Service testado     | Cenários cobertos                                                                     |
|---------------------|---------------------------------------------------------------------------------------|
| AuthService         | Email duplicado no cadastro e login com credenciais inválidas                         |
| EventService        | Data final inválida, status inválido, organizador inválido e edição por outro usuário |
| RegistrationService | Evento inexistente e inscrição duplicada                                              |
| CommentService      | Evento inexistente e remoção por usuário que não é autor                              |
| ReactionService     | Evento inexistente e reação duplicada                                                 |

Além dos testes locais, foi criado o workflow `.github/workflows/ci.yml`. Ele executa restauração de dependências, compilação em modo Release e testes automatizados em pushes para `develop`, `main` e pull requests. Assim, cada alteração enviada ao repositório passa por uma verificação mínima antes de ser integrada.

## 13 Conclusão

A Academic Events API atende todos os requisitos obrigatórios do trabalho final: autenticação JWT, cinco entidades com relacionamentos, banco PostgreSQL via EF Core, arquitetura em cinco projetos separados, DTOs request/response, services com regras de negócio, repositories com repository pattern, Swagger/OpenAPI e versionamento via Git.

Além dos requisitos mínimos, o projeto implementou:

- Cinco exceções customizadas com semântica clara de negócio
- Filtragem de eventos por status e por organizador
- Validações em duas camadas (service + banco) para evitar dados inconsistentes
- Testes automatizados com xUnit e Moq para os services principais
- Workflow de CI com restore, build e testes automatizados
- Documentação XML em português em todos os arquivos
- Diagramas C4 nos quatro níveis (contexto, container, componente e código)
- Roteiro completo de testes manuais com cenários de sucesso e de erro

O projeto demonstra domínio prático dos conceitos de C# e ASP.NET Core abordados no módulo, resultando em uma API funcional, bem organizada e pronta para ser consumida por qualquer aplicação cliente.